

Parser EBNF Generation Limitations

<https://gemini.google.com/app/2957323c5bab3272>

User prompt: Could manual recursive descent parser written in Java generate EBNF for language reference as a side effect?

Response: **TL;DR: Yes, but with major limitations.** A running parser can trace execution paths to log the EBNF rules used for a *specific input*, but it cannot automatically reconstruct the *complete* language grammar purely as a runtime side effect without inspecting or annotating the parser's code structure.

How It Works vs. The Limitations

1. The Dynamic Tracing Approach (Runtime Side Effect)

If you add logging inside your parsing methods (e.g., `parseStatement()`, `parseExpression()`), the parser can construct an EBNF trace of the paths taken during that specific parse run.

- **Pros:** Easy to implement via logging or method decorators.
- **Cons:** You only generate EBNF for the rules executed by that test input. It will miss unreached branches, optional tokens, and alternative syntax choices.

2. The Static/Structural Approach

Because a manual recursive descent parser encodes grammar rules as imperative Java logic (`if`, `while`, `switch` statements), runtime code execution doesn't inherently "know" the full set of grammatical possibilities.

To extract a complete, static EBNF reference automatically, you usually need one of the following:

- **Custom Annotations & Reflection:** Annotating Java parse methods with metadata (e.g., `@Rule(name="Expression", ebnf="Term { ('+' | '-') Term }"`) and generating the documentation via a standard Java annotation processor (`javac`).
- **AST/Grammar Objects:** Representing grammar rules explicitly as data structures inside Java rather than pure procedural control flow.
- **Static Analysis:** Parsing the Java source code of your parser to translate `while` loops into `{ ... }` (repetitions) and `if` statements into `[ ... ]` (optionals).

Summary

If your goal is an accurate **Language Reference**, static annotations or declaring a single source of truth (like an ANTLR/BNF grammar that generates the Java code) is significantly more reliable than attempting dynamic runtime extraction.

User prompt: Annotations?

Response: **TL;DR:** Yes, using Java annotations allows you to keep the EBNF definition right next to the corresponding parsing method, then extract the complete grammar automatically at build time using Java's Annotation Processing API (`javax.annotation.processing`).

How to Set It Up

1. Define the Annotations

Create custom annotations to represent grammar rules, options, and tokens:

```
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.METHOD)
public @interface GrammarRule {
    String name();
    String ebnf(); // E.g., "Term { ('+' | '-') Term }"
    String description() default "";
}
```

2. Annotate Your Recursive Descent Methods

Attach the annotation directly to your manual parsing methods:

```
public class MyParser {

    @GrammarRule(
        name = "Expression",
        ebnf = "Term { ('+' | '-') Term }",
        description = "Evaluates addition and subtraction."
    )
    public Expr parseExpression() {
        Expr left = parseTerm();
        while (match(TokenType.PLUS, TokenType.MINUS)) {
            Token op = previous();
            Expr right = parseTerm();
            left = new BinaryExpr(left, op, right);
        }
        return left;
    }
}
```

3. Generate the Reference (Build Time)

Write a custom **Annotation Processor** (`AbstractProcessor`). During compilation (`javac`), it scans your codebase for `@GrammarRule`, collects all EBNF strings, and writes them out to a single `.ebnf` or Markdown/HTML document.

Pros & Cons

Pros	Cons
Complete Coverage: Collects all rules, even unexecuted code paths.	Manual Sync: If you change the Java logic, you must manually update the annotation's <code>ebnf</code> string.
Build-Integrated: EBNF docs update automatically every time you compile.	No Logic Guarantee: The compiler cannot verify if the <code>ebnf</code> string perfectly matches the actual imperative Java parsing logic.

User prompt: Can same annotations be used for test generation?

Response: **TL;DR: Yes, partially, but with a catch.** Annotation strings alone (e.g., `"Term { ('+' | '-') Term }"`) lack the structured data needed to generate tests. To auto-generate valid syntax tests, you must either write a full **EBNF parser** inside your annotation processor to process those strings or replace simple text strings with **structured annotation parameters**.

2 Approaches to Test Generation

Approach A: Parse the EBNF String (String-based)

Keep the simple `@GrammarRule(ebnf = "...")` string, but build an EBNF parser into your Annotation Processor/Test Suite.

1. Your test generator reads the `ebnf` string.
 2. It parses the string into a grammar tree (terminals vs. non-terminals).
 3. It performs random walking or boundary coverage to output token streams (valid strings for property-based testing).
- **Downside:** You have to write an EBNF parser just to read your annotations.

Approach B: Structured Annotations (Type-Safe)

Break down the annotation elements so Java handles the structure natively:

```
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.METHOD)
public @interface GrammarRule {
    String name();
    String[] produces(); // e.g., {"Term + Term", "Term - Term", "Term"}
    String[] invalidSamples() default {}; // For negative testing
}
```

Annotated Method:

```
@GrammarRule(
    name = "Expression",
    produces = {
        "1 + 2",
        "a - b + c"
    },
    invalidSamples = {
        "1 +",
        "+ 2"
    }
)
public Expr parseExpression() { ... }
```

During build time, your annotation processor collects these samples and generates JUnit tests that feed `produces` to `parseExpression()` (expecting success) and `invalidSamples` (expecting syntax errors).

Limitations to Watch Out For

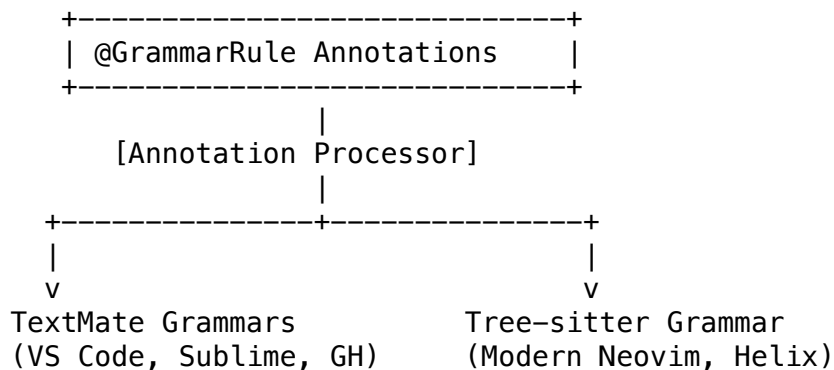
- **Context-Free Limits:** Pure EBNF cannot express context-sensitive rules (e.g., *"variable must be declared before use"* or *"type matching"*). Generated tests will test **syntax**, not **semantics**.
- **Infinite Recursion:** If your generator randomly walks recursive EBNF rules (e.g., `Expr -> Expr '+' Expr`), it can enter infinite loops unless you set a maximum recursion depth.

User prompt: Would it be possible to generate other syntax highlighter definitions from the same `ebnf` declarations?

Response: **TL;DR: Yes, absolutely.** Since EBNF defines the structure of tokens and grammar rules, an Annotation Processor or CLI tool can parse your `@GrammarRule` declarations and transform them into syntax highlighting definitions for editors like VS Code, Sublime Text, or Tree-sitter.

How the Transformation Works

Syntax highlighters rely on two primary paradigms: **Regex-based** (VS Code TextMate, Sublime) and **AST-based** (Tree-sitter).



1. Generating TextMate Grammars (VS Code, TextMate, Sublime)

TextMate grammars (`.tmLanguage.json`) use regex patterns mapped to scope names (e.g., `keyword.control`, `string.quoted`).

Mapping EBNF to TextMate

EBNF Element	EBNF Example	TextMate Equivalent (<code>.json</code>)
Terminal / Keyword	<code>'if' 'else'</code>	<code>match: "\b(if else)\b"</code>
Repetition <code>{ }</code>	<code>{ Digit }</code>	<code>match: "[0-9]+"</code>
Enclosed Block	<code>''' { Character } '''</code>	<code>begin: "\"", end: "\""</code>

How to Implement

1. Annotate terminal rules explicitly with highlighting scopes:

```
@GrammarRule(
  name = "Keyword",
  ebnf = "'if' | 'else' | 'while'",
  scope = "keyword.control"
)
```

2. Your generator iterates through rules marked with scope, extracts the literals or regexes, and constructs the TextMate JSON payload.

2. Generating Tree-sitter / Lezer (Modern Highlighters)

Modern editors (Neovim, Helix, Zed) use **Tree-sitter**, which uses actual context-free grammars written in JavaScript (`grammar.js`) rather than pure regex.

Because Tree-sitter uses structural production rules, EBNF maps almost 1:1 to it:

EBNF Syntax	Tree-sitter JavaScript Equivalent
`RuleA`	RuleB`
{ RuleA }	repeat(\$.RuleA)
[RuleA]	optional(\$.RuleA)

Your processor can output a `grammar.js` file directly from your EBNF AST.

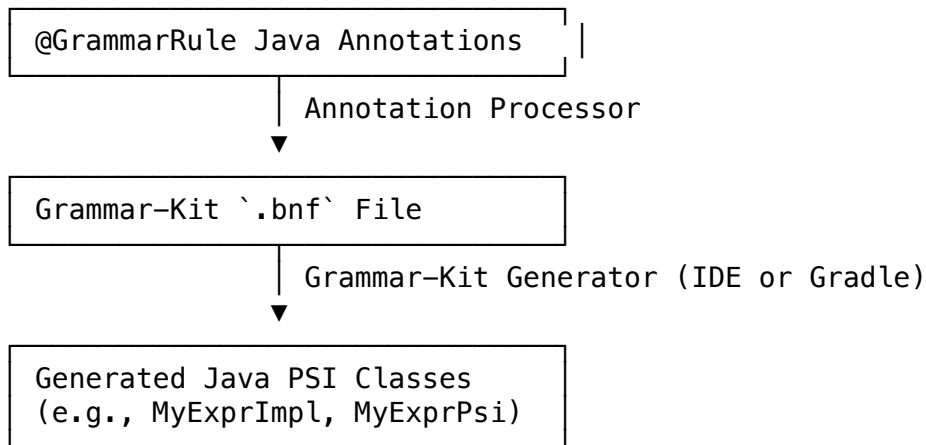
Key Caveats

- **Lexical vs. Syntactic Rules:** TextMate highlighters only care about *tokens* (keywords, numbers, strings, comments), not complex nested grammar structures. You need a way in your annotations to distinguish simple lexical tokens from structural grammar rules.
- **Statefulness:** EBNF is context-free, but some highlighting needs state (e.g., interpolated strings or multi-line block comments). You may need extended properties on your annotations (e.g., `isMultiline = true`) for complex rules.

User prompt: Could this also be used to generate IntelliJ idea PSI rules?

Response: **TL;DR: Yes, target outputting JetBrains Grammar-Kit files (.bnf).** IntelliJ Platform doesn't build PSI (Program Structure Interface) directly from raw EBNF; instead, it uses a DSL called **Grammar-Kit**. You can use your custom Java annotations to generate `.bnf` files, which Grammar-Kit then compiles into Java PSI classes (`PsiElement`, `PsiFile`, element types, and interfaces).

The Generation Pipeline



1. What the Generated .bnf File Looks Like

JetBrains' Grammar-Kit uses an EBNF-like syntax augmented with metadata headers and rule attributes:

```
{
  parserClass="com.example.plugin.parser.MyLanguageParser"
  extends="com.intellij.extapi.psi.ASTWrapperPsiElement"

  psiClassPrefix="My"
```

```
psiImplClassSuffix="Impl"  
psiPackage="com.example.plugin.psi"  
psiImplPackage="com.example.plugin.psi.impl"  
elementTypeHolderClass="com.example.plugin.psi.MyTypes"  
}
```

```
// Generated from @GrammarRule annotations  
root ::= statement_*
```

```
statement ::= expression_statement | if_statement
```

```
expression_statement ::= expression ';' { pin=1 }
```

2. Extending Annotations for PSI Metadata

For Grammar-Kit to generate clean PSI interfaces, you need additional IntelliJ-specific traits (like recovery rules, pinning, or custom interfaces) in your annotations:

```
@Retention(RetentionPolicy.SOURCE)  
@Target(ElementType.METHOD)  
public @interface PsiRule {  
    String name();  
    String ebnf();  
  
    // Grammar-Kit attributes  
    int pin() default -1; // Prevents cascading error reports  
    String extendsRule() default ""; // Hierarchical PSI inheritance  
    String[] implementsInterface() default {}; // e.g., PsiNamedElement  
}
```

3. Key Differences Between Pure EBNF and PSI Rules

While standard EBNF only cares about grammar validity, a **PSI tree** requires specific annotations and traits for IDE features:

Feature	Standard EBNF	IntelliJ Grammar-Kit (.bnf)	Why It Matters for PSI
Error Recovery	Fails on syntax error	pin=1, recoverWhile=...	Keeps IDE autocompletion & syntax highlighting working while the user is actively typing invalid code.
Node Folding	Keeps all AST nodes	private rules	Injects structural nodes without bloating the PSI tree with useless wrapper elements.
Left Recursion	Causes infinite loops	left rules (e.g., binary expressions)	Automatically handles operator precedence trees (a + b + c).

Feature	Standard EBNF	IntelliJ Grammar-Kit (.bnf)	Why It Matters for PSI
References	Ignores semantics	implements="com.intellij.psi.PsiNamedElement"	Connects variable definitions to usages for "Go to Definition" and Refactoring.

Practical Recommendation

Instead of hand-rolling code generation logic for raw IntelliJ PSI classes (which require implementing heavy platform interfaces like `PsiElement`, `ASTNode`, and `IElementType`), **use your annotation processor to emit Grammar-Kit .bnf files**. Then, run the official `gradle-grammar-kit-plugin` to generate the final Java/Kotlin PSI classes.

Sources

1. [Implementing Parser and PSI | IntelliJ Platform Plugin SDK](#)
2. [JetBrains/Grammar-Kit: Grammar files support & parser/PSI generation for IntelliJ IDEA - GitHub](#)

User prompt: What about eclipse xtext?

Response: **TL;DR: Yes, you can target Eclipse Xtext grammar files (.xtext) with your annotation processor**. However, Xtext is architected in reverse from your manual parser approach: Xtext expects its `.xtext` file to be the **primary single source of truth**, from which it auto-generates the EMF Ecore AST, LL(*) ANTLR parser, validation rules, and full Eclipse/LSP editor support.

If you reverse this flow by emitting `.xtext` files from Java annotations, you unlock rich tooling—but you must account for Xtext's unique AST-binding syntax.

1. What Generated .xtext Looks Like

Xtext rules do not just describe concrete syntax—they simultaneously define how the concrete syntax instantiates **EMF (Eclipse Modeling Framework) Ecore objects**.

```
grammar org.example.MyDsl with org.eclipse.xtext.common.Terminals
```

```
generate myDsl "http://www.example.org/MyDsl"
```

```
// Generated from @GrammarRule
Expression returns Expr:
    Term ({BinaryExpr.left=current} op=('+'|'-') right=Term)*;
```

```
Term returns Expr:
    value=INT | '(' Expression ')';
```

2. Extending Annotations for Xtext EMF Binding

Standard EBNF strings are not enough for Xtext because Xtext needs to know how token values assign to class properties (`feature=...`) and how tree nodes are created (`{TypeName}`).

```
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.METHOD)
public @interface XtextRule {
```

```
String name();  
String ebnf(); // Xtext-flavored EBNF  
String returnsType() default ""; // EMF Class return type  
}
```

Annotated Method:

```
@XtextRule(  
    name = "Statement",  
    returnsType = "Stmt",  
    ebnf = "name=ID '=' expr=Expression ';'")  
)  
public Stmt parseStatement() { ... }
```

3. What You Get vs. Structural Mismatches

What Xtext Tooling Gives You	The Impedance Mismatch
Full LSP Server: Auto-generates a Language Server Protocol backend for VS Code, Neovim, and Eclipse.	Double-Parser Problem: Xtext will generate its <i>own</i> ANTLR parser. You end up maintaining two separate parsers (your Java recursive descent parser and Xtext's generated parser).
Rich IDE Features: Syntax highlighting, code completion, cross-referencing, and outline views automatically.	EMF Coupling: Xtext heavily relies on EMF Ecore models. If your annotations don't explicitly declare structural assignments (+, =, ?=), Xtext will fail to build its AST.

Practical Recommendation

- **If your manual Java parser is mandatory:** Generating `.xtext` files via annotations is a viable bridge to get VS Code LSP support or Eclipse plugin integration without rewriting your existing codebase.
 - **If you are building a new language stack:** It is usually better to write the `.xtext` file *first* as the single source of truth, and let Xtext generate the parser and code infrastructure for you.
-